

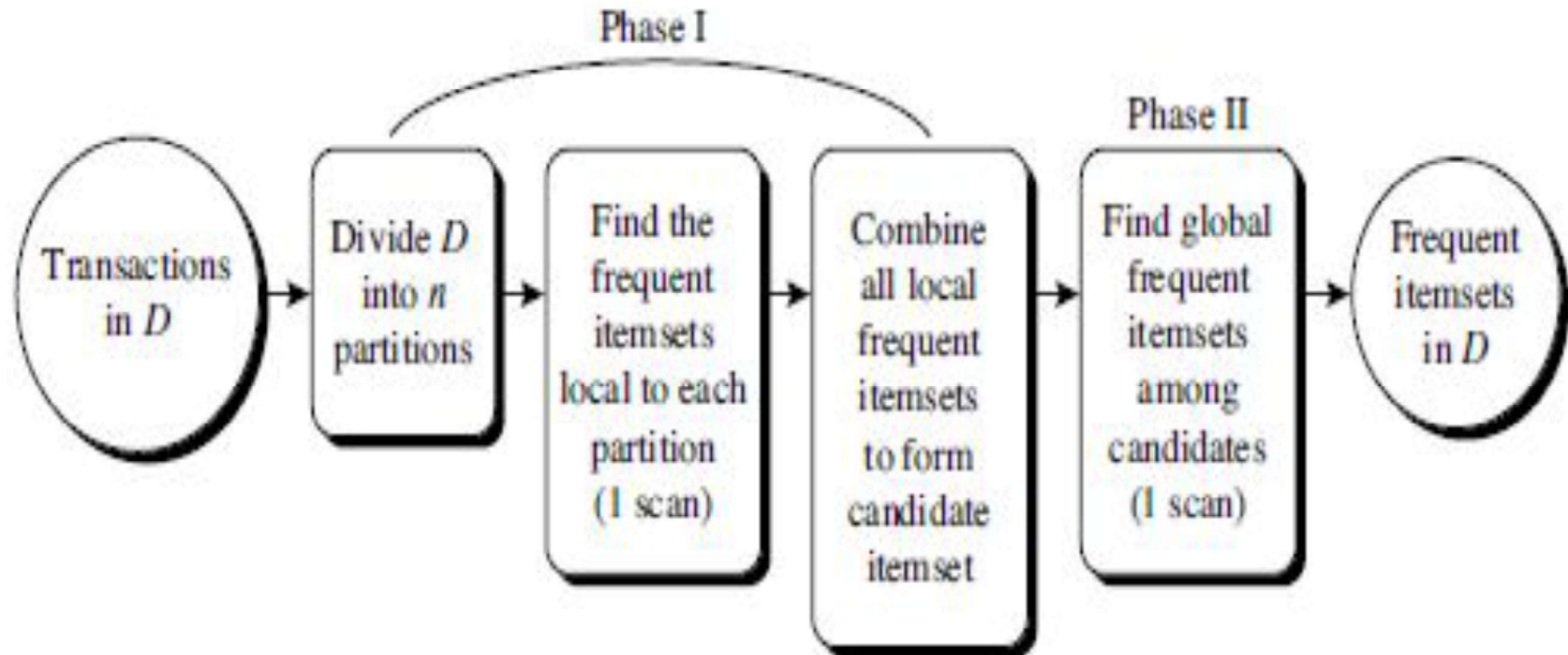
Outline

- Basic Concepts
- Frequent Item Set Mining Methods
 - Apriori: A Candidate Generation-and-Test Approach
 - Improving the efficiency of Apriori
 - FP-growth
- Strengths and Weaknesses

Further Improvement of the Apriori Method

- Major computational challenges
 - Multiple scans of transaction database
 - Huge number of candidates
 - Tedious workload of support counting for candidates
- Improving Apriori: general ideas
 - Reduce passes of transaction database scans
 - Shrink number of candidates
 - Facilitate support counting of candidates

Partition: Scan Database Only Twice



Partition: Scan Database Only Twice

- Requires two database scans to mine the frequent items.
- It consists of two phases: In phase I, the algorithm divides the transactions of D into n non overlapping partitions.
- If the minimum relative support threshold is min_sup , then the minimum support count for a partition is $\text{min_sup} * \text{the number of transactions}$ in that partition.
- For each partition, all the local frequent item sets are found.
- A local frequent itemset may or may not be frequent with respect to the entire database.

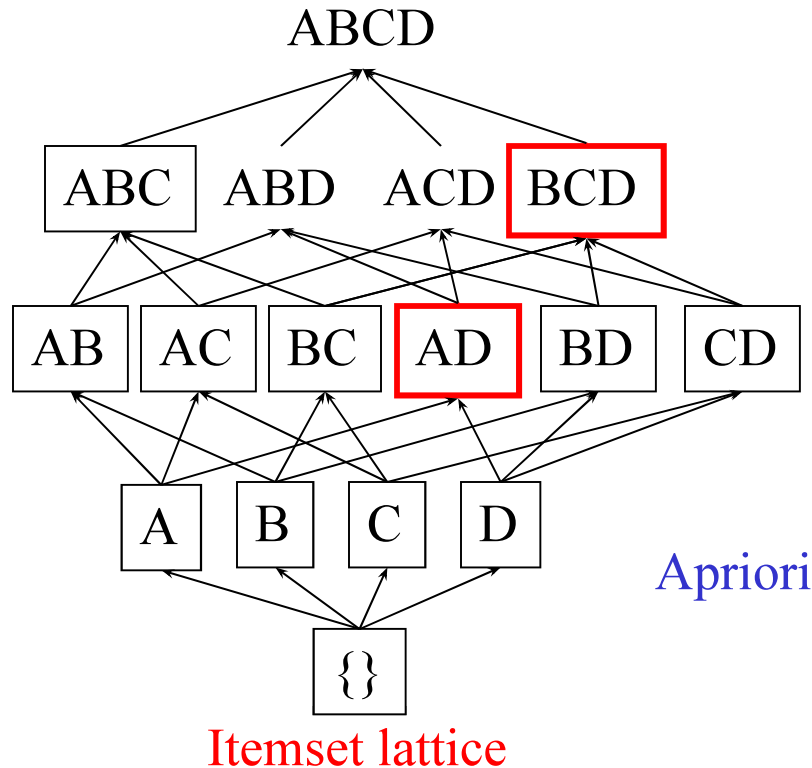
Partition: Scan Database Only Twice

- However, any itemset that is potentially frequent in D must occur as a frequent itemset in at least one of the partitions.
- Therefore, all local frequent item sets are candidate item sets with respect to D . The collection of frequent item sets from all partitions form the global candidate item sets concerning D .
- In phase II, the second scan of D is conducted, in which the actual support of each candidate is assessed to determine the global frequent item sets.

Sampling for Frequent Patterns

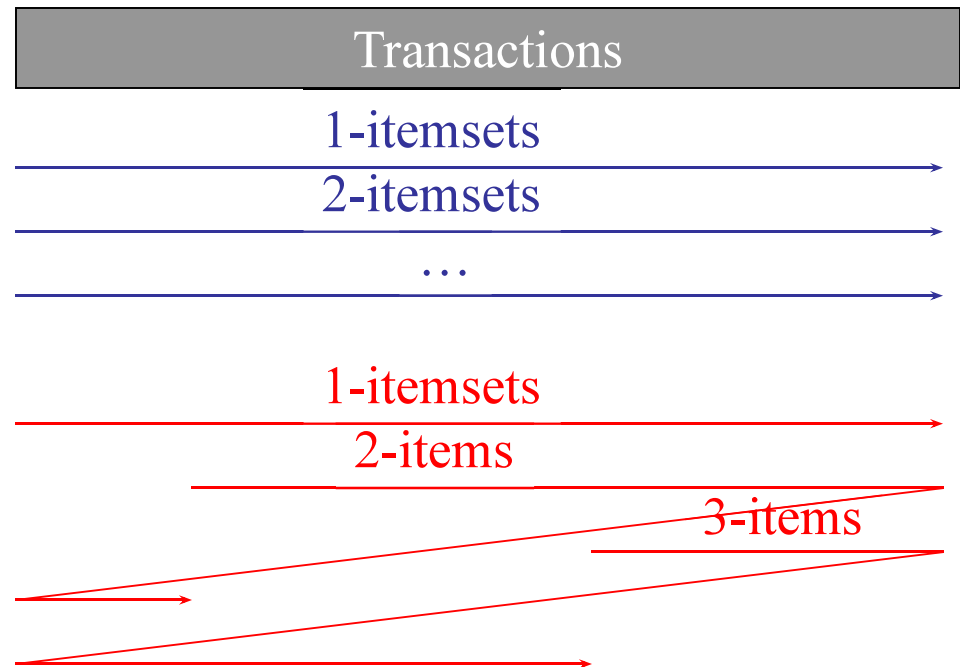
- Select a sample of original database, mine frequent patterns within the sample using Apriori.
- Scan the database once to verify frequent itemsets found in the sample; only the *borders* of the closure of frequent patterns are checked
 - Example: check *abcd* instead of *ab, ac, ..., etc.*
- Scan database again to find missed frequent patterns
- H. Toivonen. Sampling large databases for association rules. In *VLDB'96*

DIC: Reduce Number of Scans



S. Brin R. Motwani, J. Ullman,
and S. Tsur. **Dynamic itemset
counting and implication rules for
market basket data.** *SIGMOD'97*

- Once both A and D are determined frequent, the counting of AD begins
- Once all length-2 subsets of BCD are determined frequent, the counting of BCD begins



DIC

DIC: Reduce Number of Scans

- **Transaction reduction** (reducing the number of transactions scanned in future iterations):
 - A transaction that does not contain any frequent k -item sets cannot contain any frequent $(k + 1)$ -item sets.
 - Such a transaction can be removed from further consideration because subsequent database scans for j -item sets, where $j > k$, will not need to consider such a transaction.

Outline

- Basic Concepts
- Frequent Item Set Mining Methods
 - Apriori: A Candidate Generation-and-Test Approach
 - Improving the efficiency of Apriori
- FPGrowth: A Frequent Pattern-Growth Approach
- Strengths and Weaknesses

Pattern-Growth Approach: Mining Frequent Patterns Without Candidate Generation

- Bottlenecks of the Apriori approach
 - Breadth-first (i.e., level-wise) search
 - Candidate generation and test
 - Often generates a huge number of candidates
- The FPGrowth Approach (J. Han, J. Pei, and Y. Yin, SIGMOD' 00)
 - Depth-first search
 - Avoid explicit candidate generation
- Major philosophy: Grow long patterns from short ones using local frequent items only
 - “abc” is a frequent pattern
 - Get all transactions having “abc”, i.e., project DB on abc: DB|abc

Pattern-Growth Approach: Mining Frequent Patterns

- FP-growth adopts a **divide-and-conquer** strategy as follows:
- First, it compresses the database representing frequent items into a frequent pattern tree, which retains the itemset association information.
- It then divides the compressed database into a set of conditional databases (a special kind of projected database), each associated with one frequent item or “pattern fragment,” and mines each database separately.
- For each “pattern fragment,” only its associated data sets need to be examined. Therefore, this approach may substantially reduce the size of the data sets to be searched.

Construct FP-tree from a Transaction Database

<u>TID</u>	<u>Items bought</u>	<u>(ordered) frequent items</u>
100	{f, a, c, d, g, i, m, p}	{c, f, a, m, p}
200	{a, b, c, f, l, m, o}	{c, f, a, b, m}
300	{b, f, h, j, o, w}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{c, f, a, m, p}

$min_support = 3$

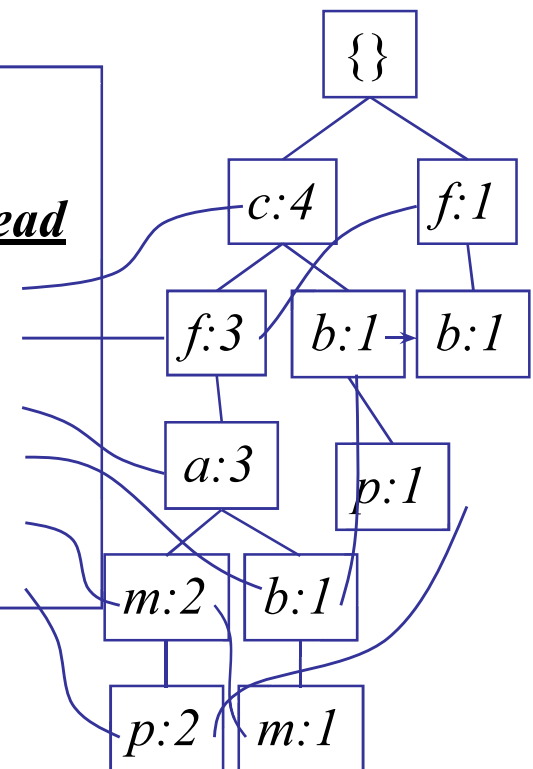
1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Sort frequent items in frequency descending order, f-list
3. Scan DB again, construct FP-tree

Header Table

Item frequency head

c	4
f	4
a	3
b	3
m	3
p	3

F-list = c-f-a-b-m-p



FP-Tree Construction

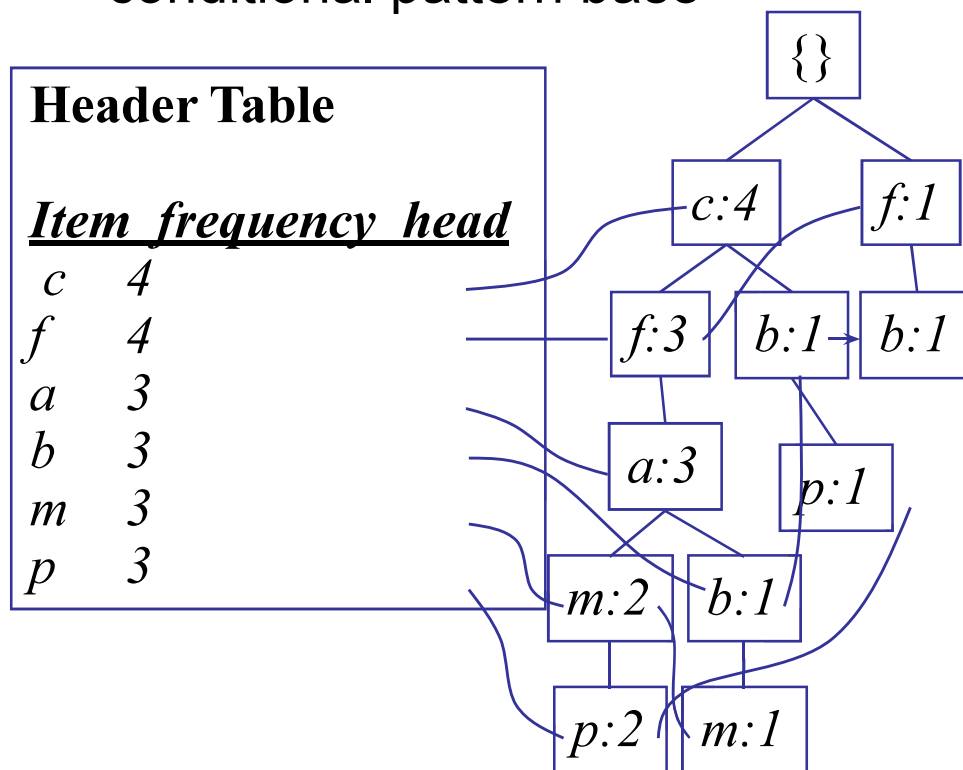
- First, create the root of the tree, labeled with “null.”
- Scan database D a second time. The items in each transaction are processed in f-list order and a branch is created for each transaction.
- For example, the scan of the first transaction (**f, a, c, d, g, i, m, p**), which contains five items (**c, f, a, m, p**), leads to the construction of the first branch of the tree with five nodes, <c: 1>, <f:1>, <a: 1>, <m: 1>, <p: 1>.
- Where **c** is linked as a child to the root, **f** is linked to **c**, and **a** is linked to **f**.
- For second transaction, the branch would share a common prefix, and increment the count. Etc.

Mining FP-Tree

- To facilitate tree traversal, an item header table is built so that each item points to its occurrences in the tree via a chain of **node-links**.
- The FP-tree is mined as follows:
 - Start from each frequent length-1 pattern (as an initial suffix pattern).
 - Construct its conditional pattern base, then construct its *(conditional) FP-tree*, and perform mining recursively on the tree.
 - The pattern growth is achieved by the concatenation of the suffix pattern with the frequent patterns generated from a conditional FP-tree.

Find Patterns Having P From P-conditional Database

- Starting at the frequent item header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item p
- Accumulate all of *transformed prefix paths* of item p to form p 's conditional pattern base



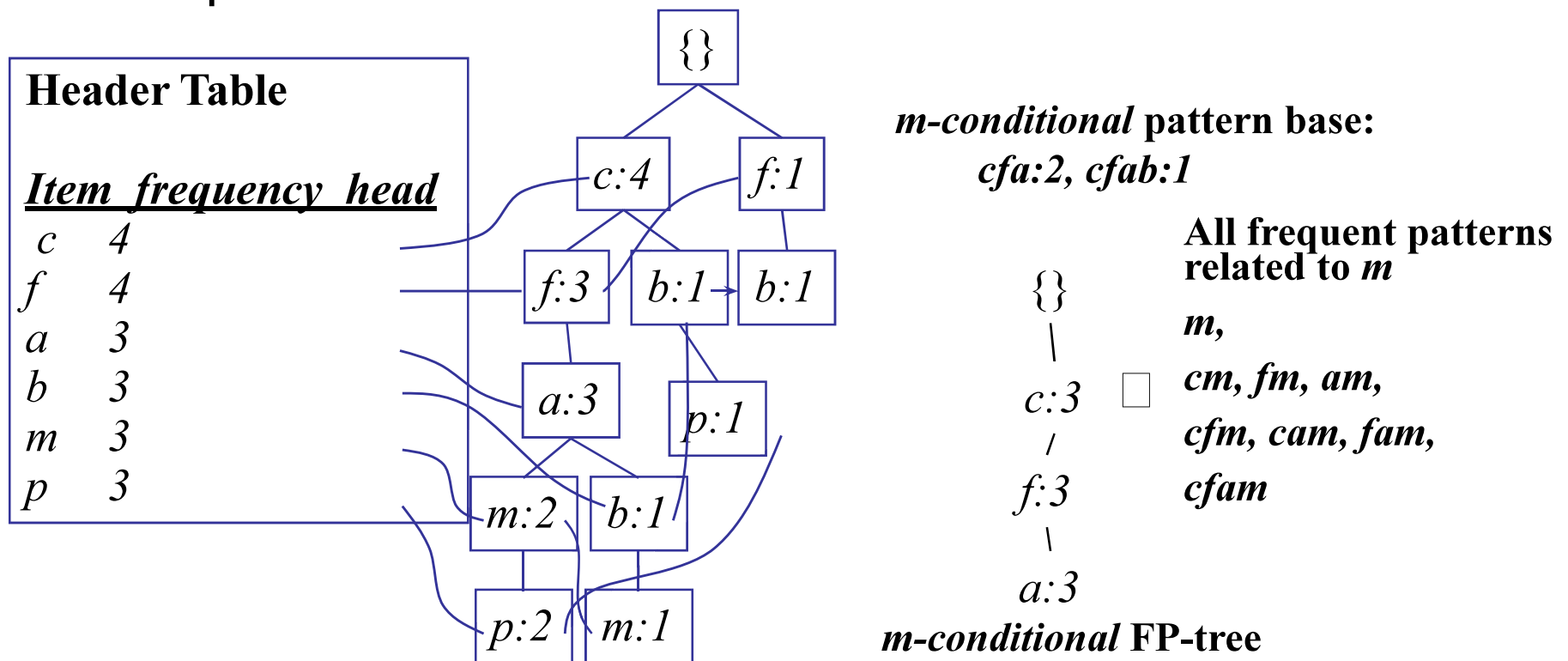
Conditional pattern bases

item cond. pattern base

f $c:3$
 a $cf:3$
 b $cfa:1, c:1, f:1$
 m $cfa:2, cfab:1$
 p $cfam:2, cb:1$

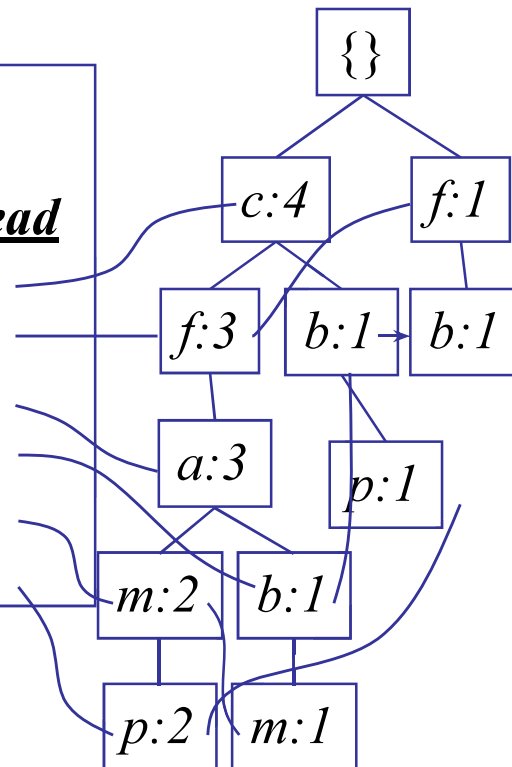
From Conditional Pattern-bases to Conditional FP-trees

- For each pattern-base
 - Accumulate the count for each item in the base
 - Construct the FP-tree for the frequent items of the pattern base



From Conditional Pattern-bases to Conditional FP-trees

Header Table		
<i>Item</i>	<i>frequency</i>	<i>head</i>
<i>c</i>	4	
<i>f</i>	4	
<i>a</i>	3	
<i>b</i>	3	
<i>m</i>	3	
<i>p</i>	3	



f-conditional pattern base:

c:3

{
|
c:3

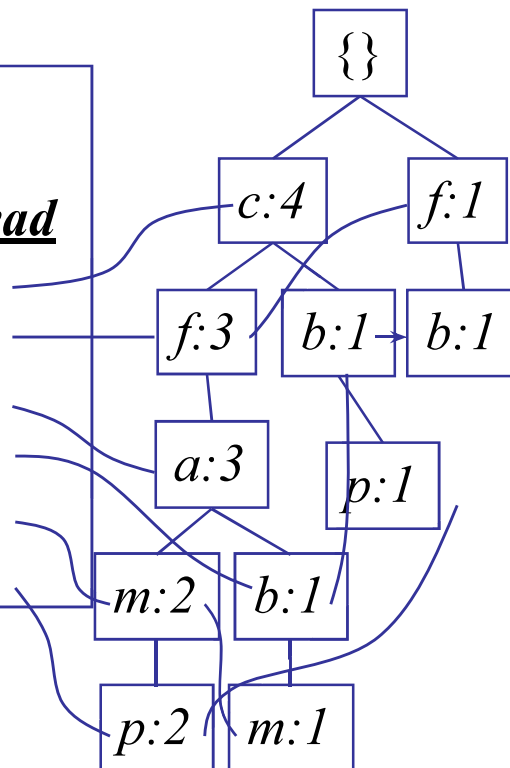
All frequent patterns
related to *c*

f,
cf

f-conditional FP-tree

From Conditional Pattern-bases to Conditional FP-trees

Header Table	
<u>Item frequency head</u>	
<i>c</i>	4
<i>f</i>	4
<i>a</i>	3
<i>b</i>	3
<i>m</i>	3
<i>p</i>	3



a-conditional pattern base:

cf:3

{}

c:3

/

f:3

□

All frequent patterns
related to *a*

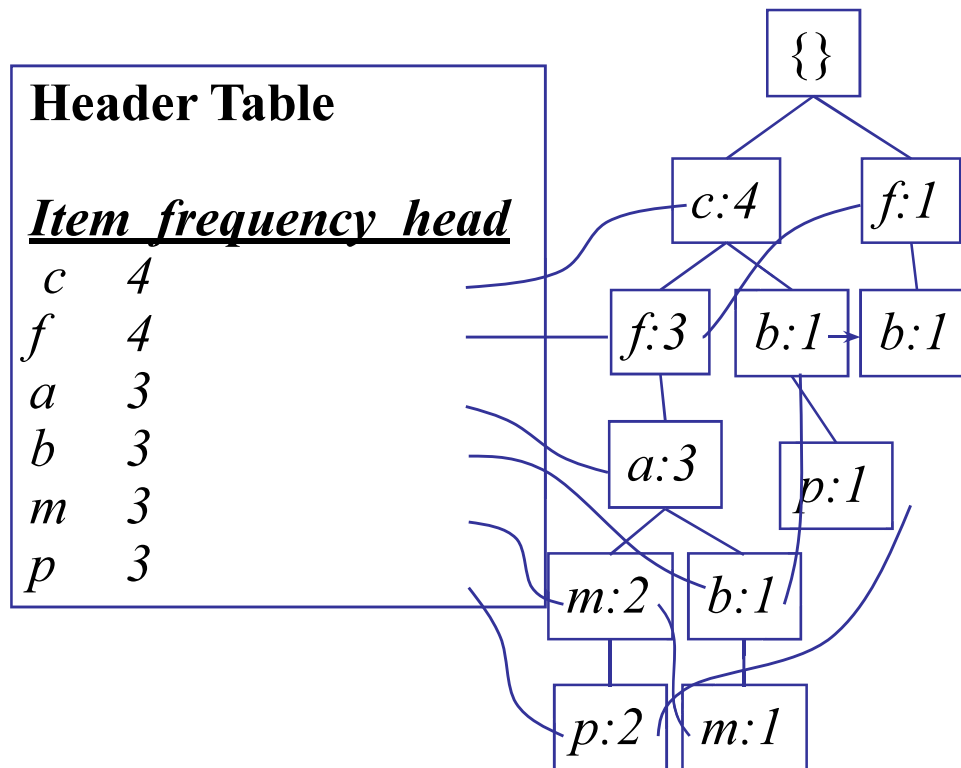
a,

ca, *fa*,

cfa

a-conditional FP-tree

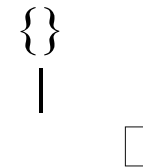
From Conditional Pattern-bases to Conditional FP-trees



b-conditional pattern base:

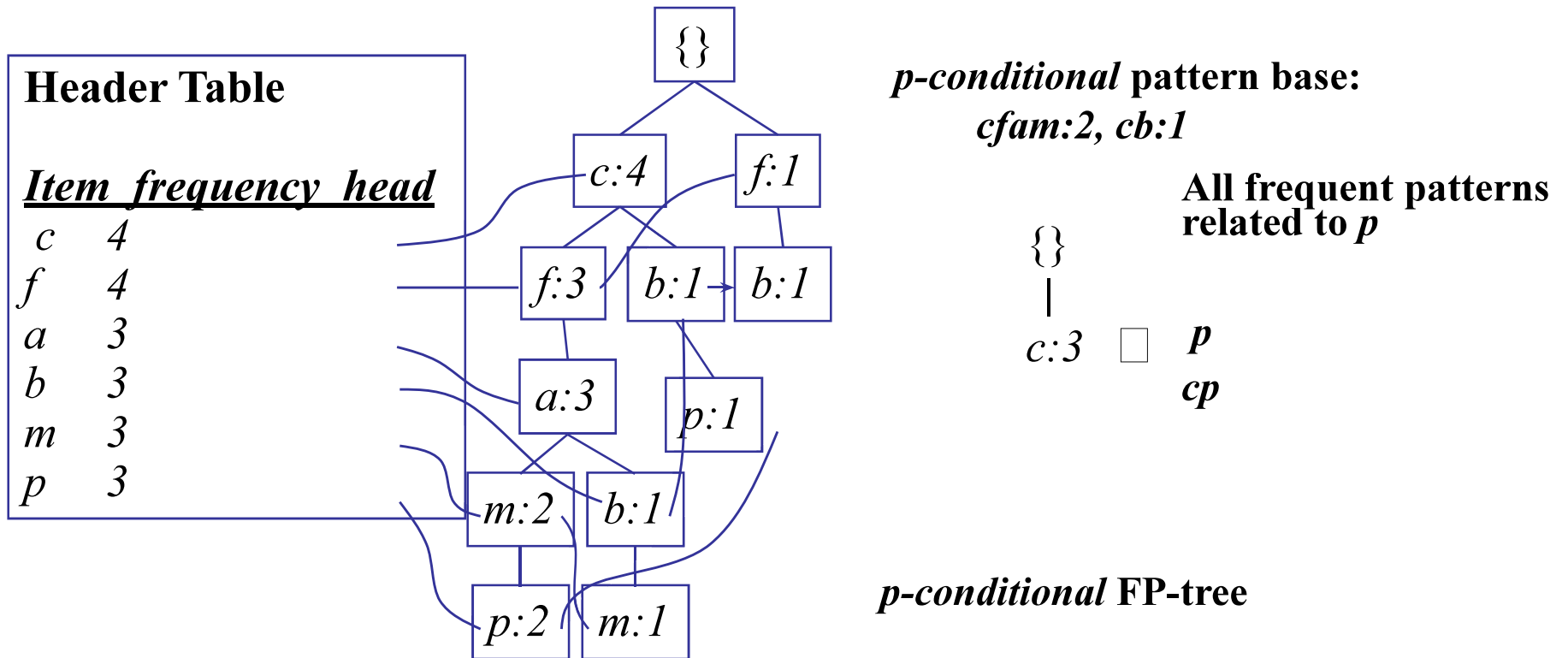
cfa:1, c:1, f:1

All frequent patterns
related to *b*

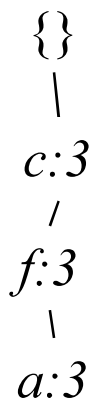


b-conditional FP-tree

From Conditional Pattern-bases to Conditional FP-trees

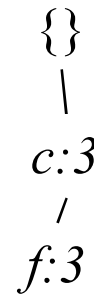


Recursion: Mining Each Conditional FP-tree



m-conditional FP-tree

Cond. pattern base of "am": (cf:3)



am-conditional FP-tree

Cond. pattern base of "fm": (c:3)



cm-conditional FP-tree

Cond. pattern base of "cam": (f:3)



cam-conditional FP-tree

Benefits of the FP-tree Structure

- Completeness
 - Preserve complete information for frequent pattern mining
 - Never break a long pattern of any transaction
- Compactness
 - Reduce irrelevant info—infrequent items are gone
 - Items in frequency descending order: the more frequently occurring, the more likely to be shared
 - Never be larger than the original database.

Advantages of the Pattern Growth Approach

- Divide-and-conquer:
 - Decompose both the mining task and DB according to the frequent patterns obtained so far
 - Lead to focused search of smaller databases
- Other factors
 - No candidate generation, no candidate test
 - Compressed database: FP-tree structure
 - No repeated scan of entire database
 - Basic ops: counting local freq items and building sub FP-tree, no pattern search and matching
- A good open-source implementation and refinement of FPGrowth
 - FPGrowth+ (Grahne and J. Zhu, FIMI'03)



Any Question